A Two-Stage Entity Resolution Pipeline with FAISS Blocking and Splink Matching

Denis Robert

August 2026

Abstract

This paper presents and evaluates a two-stage entity resolution pipeline for incomplete Canadian person records. A synthetic reference population is represented with normalized `all-MiniLM-L6-v2` embeddings and persisted in a FAISS `IndexFlatIP` store. Each query is first blocked to its top 20 vector neighbors, after which Splink performs interpretable probabilistic linkage using Jaro–Winkler comparisons for names and address, a dedicated email comparison, and a date-of-birth comparison that accounts for date structure. The approach is similar in overall architecture to the FAISS-plus-linkage workflow described by Strojny and Beręsewicz [6], although the present pipeline was independently derived and uses a different implementation and evaluation design. The implementation supports persisted-index reload without re-embedding the reference population and independently models 30% missingness for address and email. In a 15,000-query experiment comprising identical, close-variation, and unrelated records, the updated pipeline achieved 99.27% accuracy, 99.48% precision, 99.43% recall, 98.96% specificity, and a 99.45% F1 score, with a mean query time of 7.25 ms in the validation environment. The paper also discusses memory scaling, operational limits, calibration of Splink parameters, and alternatives to linear-scan FAISS indexing.

1 Introduction

Entity resolution attempts to determine whether two records refer to the same real-world entity. This task is difficult when records contain typographical variation, missing values, stale addresses, or inconsistent contact information. Comparing every query against every reference record is also unnecessarily expensive: for N reference records, exhaustive comparison requires $O(N)$ candidate comparisons per query.

The implementation in the entity-resolution repository addresses this problem with a retrieval-and-linkage architecture. A dense vector index [9] performs approximate or exact nearest-neighbor retrieval, reducing the candidate set to the closest k records. This use of dense retrieval for blocking follows a line of work including DeepER [5], pre-trained embedding comparisons [4], and universal dense blocking [3]. Splink then evaluates those candidates with explicit field comparisons and returns a match only when its probability exceeds a configurable threshold.

The representation choice is informed by recent work comparing tabular representation approaches. Vogel et al.'s *Towards Universal Tabular Embeddings: A Benchmark Across Data Tasks* (arXiv:2604.21696v1, `https://arxiv.org/abs/2604.21696v1`) [8] benchmark embeddings at cell, row, column, and table levels and show that performance depends on the downstream task and representation level; in the relevant current evaluation, textual embeddings outperform the dedicated tabular alternatives for retrieval. Franz et al.'s *Universal Embeddings of Tabular Data* (arXiv:2507.05904v1, `https://arxiv.org/abs/2507.05904v1`) [7] describes a different strategy

that transforms tabular data into a graph, learns entity embeddings with graph auto-encoders, and aggregates them into row embeddings. This graph-based approach may bear fruit for entity resolution in future experiments because it is designed to capture relationships among tabular entities and to embed unseen samples composed of similar entities. The present system therefore uses a row-level textual representation as its current baseline while leaving room to replace the embedding engine as these alternatives become more capable.

2 System Objective and Data Model

The reference population contains 50,000 synthetic Canadian person records. Each record has:

- first name;
- last name;
- date of birth;
- a Canadian address; and
- an email address.

Address and email are independently removed with a configurable default probability of 0.30. Consequently, first name, last name, and date of birth are the most consistently available identity attributes, while address and email are useful but incomplete evidence. Addresses are also treated as less stable because people move and formatting conventions change.

A record is serialized into a structured text string:

```
First Name: ...
Last Name: ...
Date of Birth: ...
Address: ...
Email: ...
```

Missing fields are omitted from the embedding text and retained as null metadata. The same person object is used for the query and for the persisted metadata, avoiding a second source of truth.

2.1 Row serialization strategies

The embedding ablation compares three textual representations of the same structured record. These representations affect only dense retrieval; Splink receives the original structured fields and uses the same comparisons in every experiment. For example, consider the record

```
first_name    = Alice
last_name     = Wong
date_of_birth = 1985-06-15
address       = 123 Main Street, Toronto, ON M5V 1A1
email         = alice.wong@example.com
```

The **default** strategy is the production representation implemented by `Person.to_text`. It uses labelled lines in identity, address, and email order:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Address: 123 Main Street, Toronto, ON M5V 1A1
Email: alice.wong@example.com
```

The **identity-first** strategy retains the same labels and values but places the optional contact fields in email-then-address order after the identity fields:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Email: alice.wong@example.com
Address: 123 Main Street, Toronto, ON M5V 1A1
```

This tests whether giving the embedding model an explicit identity prefix changes retrieval, without changing the vocabulary or the underlying data. The **compact** strategy removes labels and joins the fixed field sequence with pipe delimiters:

```
Alice|Wong|1985-06-15|123 Main Street, Toronto, ON M5V 1A1|alice.wong@example.com
```

For a record with missing address and email, the labelled strategies omit those lines, whereas compact serialization preserves field positions with empty values:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
```

```
Alice|Wong|1985-06-15||
```

The default strategy is the baseline used by the resolver and confusion-matrix experiment. Identity-first and compact are evaluation alternatives, not separate linkage models. Comparing them under the same FAISS index type, blocking sizes, Splink configuration, queries, and thresholds isolates the effect of row representation on candidate recall and downstream matching.

3 Two-Stage Algorithm

3.1 Offline generation and indexing

The offline process in `generate_data.py` performs the following steps:

1. Generate the reference population with a fixed random seed when reproducibility is required.
2. Convert every person into its textual row representation.
3. Compute a dense vector with `sentence-transformers/all-MiniLM-L6-v2` through `HuggingFaceEmbeddings`.
4. L2-normalize each vector and add it to a FAISS `IndexFlatIP` index. Inner product on normalized vectors is cosine similarity.

5. Persist the FAISS index as `people.faiss` and the person records, model name, and normalization setting as `people.json`.

The metadata file is deliberately separate from the FAISS binary index. FAISS stores the vectors and their positional order; the JSON file stores the record associated with each position. Loading reconstructs the embedding client and reuses the saved vectors without re-embedding the reference population.

3.2 Online blocking

Given a query record q, the online process embeds the same textual representation and searches the index for the top k records. The default is $k = 20$. If $e(q)$ and $e(r)$ are normalized query and reference vectors, the blocking score is

$$s_{\text{FAISS}}(q, r) = e(q)^T e(r),$$

which is cosine similarity in the range $[-1, 1]$. This stage is optimized for recall: the true match must be included in the candidate set for the second stage to recover it. FAISS [9] provides this nearest-neighbour search.

3.3 Probabilistic linkage

The candidate set is passed to Splink as a small deduplication problem containing the query and the retrieved records. Splink implements a scalable form of probabilistic linkage grounded in the Fellegi–Sunter framework [1, 2]. The configured comparisons are:

Table 1: Field comparisons and intended evidence priority.

Field	Comparison	Intended role
First name	Jaro–Winkler thresholds	High-priority identity evidence
Last name	Jaro–Winkler thresholds	High-priority identity evidence
Date of birth	Date-of-birth comparison	Strong, stable evidence
Email	Email comparison	Medium-priority evidence when present
Address	Jaro–Winkler thresholds	Lower-priority, changeable evidence

Splink produces a match probability from the comparison vector and its m/u parameters. The resolver selects the highest-scoring candidate involving the query and returns it only when

$$P(\text{match} \mid \text{comparisons}) \geq \tau,$$

where τ is the configured match threshold. Missing values are not treated as negative evidence; they provide no comparison agreement and the remaining fields determine the result.

The implementation uses FAISS for blocking and Splink 4.x inference for linkage. The current settings provide comparison ordering and thresholded similarity levels. In a production deployment, Splink's m/u probabilities should be trained on representative labelled pairs rather than relying on defaults.

4 Persistence and Operational Workflow

The workflow has two explicit commands:

```
python scripts/generate_data.py --count 50000 --missing-rate 0.3 --output-dir data
python scripts/search_cli.py --index-dir data --input-json query.json --threshold 0.85
```

The first command is an offline or scheduled job. The second command loads the persisted index, retrieves 20 candidates, performs Splink linkage, and emits either a match with both scores and the matched metadata or a no-match decision. This separation prevents expensive population embedding from occurring for every query and makes the index a deployable artifact.

5 FAISS Applicability and Memory Bounds

The 50,000-record artifact generated by this implementation provides a concrete sizing baseline. On disk, the measured `people.faiss` file is 76.8 MB and the `people.json` metadata file is 10.5 MB, for a combined 87.3 MB. This corresponds to approximately 1,536 bytes per normalized 384-dimensional float32 vector, 210 bytes per serialized person record, and 1,746 bytes per record including both artifacts. The index dimension follows the MiniLM embedding model used by the baseline.

The following extrapolation assumes the same embedding dimension, float32 storage, flat inner-product index, average metadata length, and one metadata record per vector. It uses binary RAM units (1 GiB = 2^{30} bytes), although the table labels them as the commonly used 16, 32, and 128 GB deployment sizes. It excludes the Python interpreter, embedding model, temporary construction buffers, allocator fragmentation, operating-system memory, and concurrent queries.

Table 2: Estimated capacity when the FAISS index and person metadata share the available RAM.

Available RAM	Raw capacity	Raw index size	Raw metadata size	Recommended capacity*
16 GB	9.84 million	14.4 GB	2.0 GB	6.89 million
32 GB	19.68 million	28.9 GB	4.1 GB	13.77 million
128 GB	78.71 million	115.3 GB	16.5 GB	55.10 million

*Recommended capacity reserves 30% of RAM for the operating system, Python process, model weights, FAISS temporary allocations, and query concurrency. The raw capacity is a sizing upper bound, not a production recommendation. During generation, peak memory can be higher because embeddings may be materialized before being added to FAISS. Building large indexes should therefore use batches and measure peak resident set size rather than relying only on final file size.

For this flat `IndexFlatIP` design, a query scans all vectors, so memory capacity and query latency are coupled to the record count. At tens of millions of records, an approximate index such as HNSW or an inverted-file/product-quantized index should be evaluated; FAISS [9] ships several such indexes. Quantization can reduce memory substantially, but may reduce blocking recall and must be measured against the downstream Splink match rate. Regardless of the FAISS index type, the metadata mapping must preserve vector-to-person row order and should be loaded with equivalent memory budgeting.

5.1 When an external vector database becomes appropriate

An external vector database is not automatically better than FAISS. For a single service instance, a stable reference population, and a dataset that fits comfortably in memory, local FAISS has fewer moving parts and avoids network latency. The decision should be based on operational bounds as well as raw capacity. With the measured baseline, 10 million records would require approximately 17.5 GB for the flat index plus metadata, while 50 million records would require approximately 87.3 GB before runtime overhead. These sizes place a single-process deployment near or beyond the recommended envelope of a 16 or 32 GB host and make a 128 GB host increasingly specialized.

An external vector database is likely to become more appropriate under any of the following conditions:

- the working set approaches roughly 70% of host RAM, leaving insufficient room for the embedding model, Python, Splink, index rebuilds, and concurrent queries;
- the reference population is large enough that flat-index scan latency or index-build time violates the service-level objective, even after testing FAISS approximate indexes;
- multiple application instances need a shared index, replicas, rolling updates, or failover without copying a large binary artifact to every host;
- records require frequent inserts, deletes, or metadata updates, making immutable local snapshots operationally expensive; or
- the system needs managed durability, access control, monitoring, backups, multi-tenant isolation, or independent scaling of storage and query capacity.

As a practical boundary, local FAISS is a strong default for the 50,000-record baseline and remains straightforward through the low millions. Between roughly 7 million and 14 million records, the 16/32 GB sizing envelope suggests benchmarking an approximate FAISS index against a managed or self-hosted vector database. Above roughly 50 million records, or whenever the index must be shared across services and regions, an external system is generally the safer operational choice. These are planning thresholds rather than algorithmic limits: compression, sharding, and hardware can move them, while a strict latency or availability requirement can move the decision earlier. The external database should still be evaluated by blocking recall at $k = 20$, not only by vector-query latency, because Splink cannot recover a true match that the retrieval layer omits.

6 Complexity and Failure Modes

Let d be the embedding dimension, N the reference population size, and k the blocking size. Building the flat FAISS index costs approximately $O(Nd)$ vector storage and embedding work. A flat inner-product query costs $O(Nd)$, although FAISS offers approximate indexes when the population grows beyond the intended scale. Splink then operates on only $k + 1$ records, making the linkage stage independent of the full population size after blocking.

The principal failure mode is blocking recall. A correct record omitted from the top k candidates cannot be recovered by Splink. Other risks include embedding representations that overemphasize address tokens, duplicate or common names, poorly calibrated Splink probabilities, and distribution shift between synthetic data and operational data. Evaluation should therefore report blocking recall separately from final precision, recall, and calibration.

7 Evaluation Plan

A useful benchmark should contain labelled positive and negative pairs, controlled missingness, realistic address changes, spelling errors, and hard negatives sharing names or dates. The following metrics should be reported:

1. blocking recall at $k \in \{10, 20, 50, 100\}$;
2. final precision, recall, F1, and false match rate at several thresholds;
3. precision–recall and calibration curves for Splink probabilities;
4. mean and tail query latency; and
5. index size and offline build time.

The benchmark should include ablations for missing email, missing address, both fields missing, address changes, and name perturbations. It should also compare row serialization strategies so that improvements are attributable to representations rather than only to the matcher.

7.1 Measured Section 7 benchmark

These requirements were implemented in `experiment_section7_eval.py` and run with 5,000 reference records, 15,000 labelled queries, a 30% independent missingness rate, and 500 records per ablation. The labelled queries consisted of identical positives, close same-entity variants, and unrelated negatives. Splink used its current untrained default m/u parameters, so the measurements are an engineering baseline rather than calibrated production probabilities. The evaluator wrote the complete results to `section7_results.json` and `section7_metrics.csv`.

Table 3 compares the three row serialization strategies. Blocking recall is measured over the 10,000 positive queries. Final linkage metrics use the production top-20 candidate set, while recall is also reported at $k = 10, 50, 100$. Latency percentiles cover embedding plus FAISS blocking per query; the reported end-to-end mean adds the batched Splink inference time divided by the number of queries.

Table 3: Measured serialization and retrieval/linkage results.

Strategy	R@10	R@20	R@50	R@100	$P_{.85}$	$R_{.85}$	$F1_{.85}$
Default	99.81%	99.84%	99.93%	99.96%	99.58%	99.80%	99.69%
Identity first	99.81%	99.86%	99.94%	99.96%	99.59%	99.82%	99.71%
Compact	99.95%	99.95%	99.97%	99.99%	99.64%	99.89%	99.77%

The compact serialization was the strongest of the three in this run, improving top-20 blocking recall from 99.84% to 99.95% and reducing the false-match rate at threshold 0.85 from 0.84% to 0.72%. The difference is descriptive rather than statistically conclusive and should be retested on labelled operational data. For the default strategy, index build time was 37.13 seconds, p95 blocking latency was 24.13 ms, and estimated mean end-to-end latency was 24.44 ms. The corresponding values for identity-first were 36.22 seconds, 25.32 ms, and 34.85 ms; compact was 28.53 seconds, 19.95 ms, and 21.22 ms.

At the default 0.85 threshold, every ablation query was accepted in this synthetic positive-only slice once it reached Splink, so the main observed degradation was retrieval: removing both optional fields reduced top-10 recall to 98.2%, although all tested ablations reached 100.0% by top

Table 4: Blocking-recall ablations at $n = 500$ per condition.

Condition	R@10	R@20	R@50	R@100
Baseline	100.0%	100.0%	100.0%	100.0%
Missing email	100.0%	100.0%	100.0%	100.0%
Missing address	99.6%	100.0%	100.0%	100.0%
Missing both	98.2%	100.0%	100.0%	100.0%
Address change	100.0%	100.0%	100.0%	100.0%
Name perturbation	99.8%	100.0%	100.0%	100.0%

20. The evaluator also produces ten-bin reliability data and threshold curves; those outputs should be recalculated after Splink is trained on labelled pairs.

8 Confusion-Matrix Experiment

The implementation includes `experiment_confusion_matrix.py`, which evaluates the current algorithm on $n = 5{,}000$ generated reference rows. For every reference person, the test creates three queries: an identical row labelled as the same entity, a close but non-identical row generated with controlled perturbations and labelled as the same entity, and an independently generated unrelated row labelled as a different entity. The latter is not present in the reference index. FAISS retrieves the top 20 candidates for every query, after which Splink applies the same field comparisons and decision threshold used by the resolver. The experiment batches the queries into one Splink link-only inference call for execution efficiency; this changes execution strategy but not the candidate pairs or scoring configuration.

Table 5: Confusion-matrix experiment metaparameters.

Parameter	Value
Reference/test rows (n)	5,000
Queries per test row	3
Total queries	15,000
Missing address/email rate	0.30
Embedding model	`all-MiniLM-L6-v2`
FAISS blocking size (k)	20
Splink match threshold (τ)	0.85
Close-row variation rate	0.15
Random seed	42

Table 6: Measured confusion matrix over 15,000 queries.

	Predicted match	Predicted no match
Actual same entity	9,943 (TP)	57 (FN)
Actual different entity	52 (FP)	4,948 (TN)

All 5,000 identical queries were correctly matched. Among the 5,000 close same-entity queries, 4,943 were matched and 57 were missed. Among the 5,000 unrelated queries, 52 were incorrectly

accepted and 4,948 were rejected. The resulting accuracy was 99.27%, precision 99.48%, recall 99.43%, specificity 98.96%, and F1 99.45%. Index construction took 37.97 seconds and batched query processing took 108.7 seconds, or 7.25 milliseconds per query on the test environment. These measurements are a baseline rather than a general performance guarantee; hardware, model caching, and Splink configuration affect both quality and latency.

8.1 Trained versus untrained linkage parameters

All figures reported so far use Splink's default, untrained m/u values together with a fixed match prior of 0.0001. Because the whitepaper's recommendations depend on the measured operating point, we quantify whether calibrating m/u changes the results materially. We reuse the 50,000-record reference index and evaluate the same three-query-per-row protocol for 2,000 rows (6,000 queries) at $\tau = 0.85$ and $k = 20$, holding the match prior at 0.0001 so that the comparison isolates the effect of m/u alone.

Two calibration schemes were considered. The first is Splink's unsupervised expectation maximisation over the reference population. Because the reference set is de-duplicated and contains essentially no true duplicate pairs, EM fits m/u from "different people who happen to share a name or date of birth," which inflates match probabilities and yields gross over-matching (F1 fell to roughly 0.71). This negative result is an important caveat: unsupervised EM on a near-duplicate-free reference is not a safe calibration strategy for streaming link-only resolution.

The second, recommended scheme calibrates each comparison level's m and u directly from labelled pairs generated by the same process as the confusion experiment: identical rows and close-variant rows are labelled matches, and independently generated unrelated rows are labelled non-matches. Empirical level probabilities are computed over these pairs with Laplace smoothing (0.5), and null levels are left to Splink's data-driven handling. This supervised calibration is defensible because it uses genuine match and non-match evidence rather than the accidental shared-field structure of the reference population.

Table 7: Trained versus untrained linkage parameters over 50,000 reference records and 6,000 queries ($\tau = 0.85$, $k = 20$, prior 0.0001).

Variant	Accuracy	Precision	Recall	Specificity	F1
Untrained (default m/u)	97.27%	96.83%	99.15%	93.50%	97.97%
Trained (supervised, labelled pairs)	91.38%	89.11%	99.20%	75.75%	93.88%

The supervised calibration is well behaved and changes the operating point in a measurable way. Recall is essentially unchanged (99.15% to 99.20%), while precision falls from 96.83% to 89.11% and specificity from 93.50% to 75.75%, giving an F1 of 93.88% versus 97.97% untrained. In absolute terms the trained variant trades a small reduction in false negatives (34 to 32) for a substantially larger increase in false positives (130 to 485). The results therefore confirm that reported figures are sensitive to how m/u are obtained, and they motivate the labelled-pair calibration and related research directions discussed in Section 9. As a rule, parameters should be trained on labelled operationally representative pairs and evaluated for calibration before reported numbers are treated as production expectations.

8.2 Large duplicate-bearing benchmark at 100K rows

The experiments above evaluate a near-duplicate-free reference. To stress the resolver under a realistic mix of true duplicates and unrelated records, `experiment_duplicate_benchmark.py` builds a 100,000-record base population and adds a near-duplicate twin (small field differences at the same 0.15 variation rate) for 3% of base records, yielding a 103,000-record reference index that genuinely contains matching records. The evaluation uses 6,000 balanced queries: 3,000 positives are fresh small-difference variants that are *not verbatim* in the index but each match a base record, and 3,000 negatives are independently generated unrelated people. Queries are blocked to the top 20 FAISS neighbours and scored with Splink at $\tau = 0.85$; only the linkage parameters differ. Three schemes are compared: the untrained default m/u with a fixed prior of 0.0001; supervised calibration of m/u from labelled duplicate/non-match pairs; and unsupervised expectation maximisation over the reference population, which here contains genuine duplicate pairs for EM to exploit.

Table 8: Large duplicate-bearing benchmark: 100,000 base records plus 3,000 duplicates (103,000-record index) and 6,000 queries ($\tau = 0.85$, $k = 20$).

Variant	Accuracy	Precision	Recall	Specificity	F1
Untrained (default m/u, prior 0.0001)	93.12%	89.59%	97.57%	88.67%	93.41%
Supervised (labelled pairs)	82.52%	74.85%	97.93%	67.10%	84.85%
Unsupervised (EM, prior 0.0071)	61.57%	56.69%	98.03%	25.10%	71.84%

In confusion-matrix terms the untrained variant produced TP = 2,927, FN = 73, FP = 340, TN = 2,660; supervised TP = 2,938, FN = 62, FP = 987, TN = 2,013; and EM TP = 2,941, FN = 59, FP = 2,247, TN = 753. Recall is comparable across all three (97.6–98.0%), but the trained schemes increasingly sacrifice precision and specificity for that recall: supervised precision falls to 74.9% and specificity to 67.1%; EM falls to 56.7% precision and a very low 25.1% specificity. The EM run also estimates a much larger match prior (0.0071 versus the fixed 0.0001), which, together with EM-fitted m/u, drives the gross over-matching. The untrained default thus yields the best F1 (93.41%) on this duplicate-bearing population, reinforcing that the choice of m/u strongly affects the measured operating point and that reported figures must be tied to the exact parameter configuration used.

8.3 Decision-threshold and address-weight sensitivity

A grid search over the decision threshold τ and the address comparison weight was run on the same 5,000-record / 15,000-query confusion-matrix population. For each address weight the pipeline was scored once (all candidate pairs emitted at threshold zero) and each candidate decision threshold applied afterwards, giving 30 operating points for a small computational cost. The two findings below concern τ (where tuning helps) and the address weight (where it does not, on this data set).

Raising the decision threshold improves F1. At the default (full-strength) address comparison, increasing τ from 0.85 to 0.95 improves the F1 while keeping accuracy essentially flat. Because recall was already near its ceiling, the higher threshold removes the weakest false positives with only a modest recall penalty.

At $\tau = 0.95$ the confusion matrix is TP = 9,930, FN = 70, FP = 4, TN = 4,996: false positives fall from 38 to 4 (precision 99.62% to 99.96%, specificity 99.24% to 99.92%) while recall falls only from 99.44% to 99.30% (F1 99.53% to 99.63%). The improvement is modest but consistent with

Table 9: F1 versus decision threshold at full address weight on the 5,000-record confusion-matrix population (15,000 queries).

τ	Accuracy	Precision	Recall	Specificity	F1
0.85	99.37%	99.62%	99.44%	99.24%	99.53%
0.87	99.30%	99.62%	99.33%	99.24%	99.47%
0.90	99.28%	99.62%	99.30%	99.24%	99.46%
0.92	99.28%	99.62%	99.30%	99.24%	99.46%
0.95	99.51%	99.96%	99.30%	99.92%	99.63%

the analysis above: in a regime where recall has headroom, the decision threshold is an effective precision lever and should be tuned on a validation set rather than fixed at 0.85.

Weakening the address comparison did not help. The intuition that address is too volatile to trust as an identity signal was tested by scaling the address agreement probabilities down ("weakening" the comparison) and re-running the sweep for strengths 0.6–0.95. On this data set the approach did not improve performance: after threshold tuning, every weakened configuration topped out at roughly 98.5% F1, well below the 99.63% reached by the full-strength address comparison at $\tau = 0.95$. The reason is that the address field here is genuinely informative: 30% of addresses are missing independently, and the addresses that do exist are long and highly discriminative, so address agreement is valuable evidence for correctly rescuing the close same-entity queries. Downgrading that evidence cost more true positives than it suppressed false ones.

This result is intentionally reported as a negative and should not be generalised. Its scope is limited to the synthetic Canadian population and its mutation model, where addresses are strongly identifying. There may be regimes in which weakening a volatile field is beneficial — for example, addresses that change frequently between legitimate records of the same entity, noisy or generic addresses (unit-only entries, post-office boxes), or populations with many address collisions in high-density buildings. Establishing whether and exactly when a weak-address weighting is viable requires controlled experiments across data sets with tunable address volatility; the implementation foresees this and `experiment_f1_sweep.py` plus `weaken_comparison` make such a study straightforward to run. That investigation is left as future work.

9 Further Research

9.1 Alternative embedding engines

The current MiniLM encoder is a useful baseline, but it is not specialized for entity resolution or structured rows. Candidate alternatives include:

- larger sentence-transformer models for stronger semantic representations;
- multilingual encoders for names and addresses across language communities;
- character n-gram or TF–IDF vectors for typo-sensitive retrieval;
- domain-adapted contrastive encoders trained on positive and hard-negative person pairs;
- graph-based universal tabular embeddings such as those proposed by Franz et al. [7]; and

- hybrid retrieval that combines dense similarity with lexical or field-specific indexes.

The comparison should measure not only final linkage quality but also whether the embedding preserves the true match in the top 20. A particularly promising approach is multi-view retrieval: learn separate views for identity fields, contact fields, and address fields, then combine their scores before Splink. This could reduce the tendency of a long address to dominate a short but highly identifying date of birth.

These directions are nevertheless worth tempering against the measured F1 budget. On this data set the default top-20 blocking recall is already 99.84% (rising to 99.95% with the compact serialization), which caps end-to-end recall. Because the measured end-to-end recall (99.43%) sits well below that ceiling, the recall that a better blocker could add is at most 0.16% (default serialization), and even a perfect blocker would raise the F1 ceiling from 99.66% to 99.74% — a gain of only about 0.08%. In other words, alternative embeddings and multi-view retrieval are unlikely to move F1 materially while top-k blocking recall is already near-saturated. Their value would instead emerge in regimes where blocking recall is the binding constraint: much larger and noisier populations, or retrieval configurations that no longer hold near-perfect recall at a small k. Absent such a regime, the higher-impact research target remains the Splink linkage stage, which currently gives up about 0.41% of recall between blocking and the final decision and is where the precision-recall trade-off can actually be tuned (for example through the threshold and m/u calibration experiments above).

9.2 Replication on non-synthetic data

Every result in this paper is measured on synthetically generated Canadian person records (Faker with a custom Canadian address provider). Synthetic data is valuable because it supplies controlled missingness, exact ground truth, and repeatable duplicate rates, but it embeds assumptions about name distributions, address formats and volatility, email conventions, and how errors arise between records of the same entity. Whether the findings hold with real data is an open empirical question, so the experiments should be replicated on non-synthetic, labelled record-linkage data. The replication should run the identical pipeline and protocols — same top-k FAISS blocking, same comparisons and decision machinery, the confusion-matrix three-query protocol, and the trained/untrained/supervised m/u, threshold, and address-weight sweeps — and compare the resulting confusion matrices, F1, and the conclusions drawn from them.

The natural test beds are public benchmark record-linkage data sets with known ground truth. These include the North Carolina voter-registration data [14, 13] (widely used in the record-linkage literature, including masked variants), the Cora citation data [11], the FEBRL restaurant data [12], the Abt–Buy, Amazon–Google, and DBLP–ACM product and citation pairs from the entity-resolution benchmarks [10], and analogous entity data. Where a data set lacks the fields used here (date of birth, address, email), the comparison should be restricted to the fields it does provide, or the pipeline should be adapted to the available schema while keeping the methodology fixed.

Such replication serves several specific purposes. First, it tests whether the address field is as informative as the synthetic data suggests: real addresses are frequently volatile (people move) or generic (unit-only entries, post-office boxes), so the finding that weakening the address comparison did not help may not transfer. Second, real data can reveal whether retraining m/u on the collection itself behaves like the lab setting, where the untrained default outperformed the trained variants, or whether genuinely noisy real duplicates make calibration beneficial. Third, the recall distributions and blocking ceiling measured at $k = 20$ in Section 9 may differ when names collide at realistic rates,

changing where the F1 budget lies between the blocking and linkage stages. Finally, replication must handle missing values, inconsistent postal/date formats, and multilingual names, which the synthetic generator controls but real data exposes. Privacy considerations apply because person records are personally identifying; labels and ground truth should be obtained from datasets that are published for research or from de-identified data with controlled access. The goal is not to certify a specific score but to confirm which qualitative conclusions — which levers move F1 and which do not — survive contact with real data.

9.3 Calibration and deployment

Future work should estimate Splink parameters from labelled operational samples, quantify probability calibration, and introduce drift monitoring for names, domains, postal formats, and address conventions. Privacy-preserving evaluation is also important: synthetic data is useful for development, but production validation must control access to personally identifying information and log only the minimum necessary evidence.

10 Conclusion

The implemented architecture combines a fast vector retrieval layer with an interpretable probabilistic linkage layer. FAISS narrows a 50,000-record population to 20 candidates, while Splink compares identity, contact, and address fields under a configurable decision threshold. Persisting the index and metadata makes the workflow operationally reusable. The most important next steps are to train and calibrate Splink parameters on labelled pairs and benchmark alternative row and tabular embedding engines using blocking recall as a first-class metric.

The measured experiments also isolate which control levers move the F1 across the validation population and which do not. Three levers had a clear effect. The decision threshold τ is the cheapest and most effective precision lever: because recall already sat near its ceiling, raising τ from 0.85 to 0.95 on the confusion-matrix population lifted F1 from 99.53% to 99.63% (precision 99.62% to 99.96%, specificity 99.24% to 99.92%) at a small recall cost. The row serialization strategy also matters: the compact representation raised top-20 blocking recall to 99.95% and F1 to 99.77%, confirming that retrieval recall is a first-order lever for end-to-end F1. So is the blocking size k, which sets an upper bound on end-to-end recall; the default $k = 20$ already achieves 99.84% blocking recall, so increasing k buys only a marginal recall gain while adding retrieval cost. Two levers did not help on this data set. Weakening the address comparison (scaling its agreement probabilities down by factors 0.6–0.95) never improved F1, topping out near 98.5% rather than the 99.63% reached at full address strength, because address agreement is genuinely informative for the close same-entity queries here. Likewise, retraining m/u on the synthetic population — whether supervised or by expectation maximisation — over-matched and lowered F1 relative to the untrained default: calibration only helps when it is driven by labelled, operationally representative pairs, which the synthetic reference does not contain.

References

[1] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. Journal of the American Statistical Association, 64(328):1183–1210, 1969. https://doi.org/10.1080/01621459.1969.10501049.

[2] Robin Linacre, Sam Lindsay, Theodore Manassis, Zoe Slade, Tom Hepworth, Ross Kennedy, and Andrew Bond. Splink: Free software for probabilistic record linkage at scale. International Journal of Population Data Science, 7(3), 2022. https://doi.org/10.23889/ijpds.v7i3.1794. Software repository: https://github.com/moj-analytical-services/splink.

[3] Tianyi Wang, Hongyu Lin, Xu Han, Xu Chen, Bo Cao, and Liang Sun. Towards universal dense blocking for entity resolution. 2024. Code repository: https://github.com/tshu-w/uniblocker.

[4] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. Pre-trained embeddings for entity resolution: An experimental analysis. Proceedings of the VLDB Endowment, 16(9):2225–2238, 2023. https://doi.org/10.14778/3598581.3598594.

[5] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. Proceedings of the VLDB Endowment, 11(11):1454–1467, 2018. https://doi.org/10.14778/3236187.3236198.

[6] Tymoteusz Strojny and Maciej Beręsewicz. BlockingPy: Approximate nearest neighbours for blocking of records for entity resolution. SoftwareX, 34:102583, 2026. https://doi.org/10.1016/j.softx.2026.102583. Software repository: https://github.com/ncn-foreigners/BlockingPy.

[7] Astrid Franz, Frederik Hoppe, Marianne Michaelis, and Udo Göbel. Universal embeddings of tabular data. arXiv:2507.05904v1, 2025. https://arxiv.org/abs/2507.05904v1.

[8] Liane Vogel, Kavitha Srinivas, Niharika D'Souza, Sola Shirai, Oktie Hassanzadeh, and Horst Samulowitz. Towards universal tabular embeddings: A benchmark across data tasks. arXiv:2604.21696v1, 2026. https://arxiv.org/abs/2604.21696v1.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3):535–547, 2019. https://doi.org/10.1109/TBDATA.2019.2921572. Preprint: https://arxiv.org/abs/1702.08734.

[10] Hanna Köpcke, Andreas Thor, and Erhard Rahm. Evaluation of entity resolution approaches on real-world match problems. Proceedings of the VLDB Endowment, 3(1–2):484–493, 2010. https://doi.org/10.14778/1920841.1920904. Data sets (Abt–Buy, Amazon–Google, DBLP–ACM, North Carolina Voters, Cora): https://dbs.uni-leipzig.de/research/projects/benchmark-datasets-for-entity-resolution/.

[11] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient clustering of high-dimensional data sets with application to reference matching. In Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pages 169–178, 2000. https://doi.org/10.1145/347090.347123. Data set (Cora): https://www.openicpsr.org/openicpsr/project/100859/version/V1/view and https://hpi.de/naumann/projects/repeatability/datasets/cora-dataset.html.

[12] Peter Christen, Tim Churches, and Markus Hegland. Febrl – A parallel open source data linkage system. In Proceedings of the Eighth Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD), volume 3056 of Lecture Notes in Artificial Intelligence, pages 638–647, 2004. FEBRL data sets: https://www.kaggle.com/datasets/adrianevensen/restaurants-entity-resolution and https://www.cecs.anu.edu.au/~Peter.Christen/Febrl.html.

[13] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In Proceedings of the 2018 International Conference on Management of Data (SIGMOD), pages 19–34, 2018. https://doi.org/10.1145/3183713.3196926.

[14] North Carolina State Board of Elections. Voter registration data (public data download). https://www.ncsbe.gov/results-data/voter-registration-data. Kaggle mirror: https://www.kaggle.com/datasets/jerimee/north-carolina-voter-file.